Domain Randomization for Sim-to-Sim Transfer

Panagiotis Kiousis Emmanouil Tsiknakis

June 22, 2025

Intelligent Control — Course Project

Contents

1 Introduction **2**

2 Domain Randomization **2**

3 Environment **2**
3.1 Environment Initialization 2
3.2 Dynamics 3
3.3 Goal Detection 3
3.4 Reward Computation 4
3.5 Agent–Environment Interaction 4
3.6 3D Visualization 5

4 Neural Network Architecture **6**
4.1 Actor–Critic Model 6
4.2 Observation Space 6

5 Training with Proximal Policy Optimization (PPO) **7**
5.1 PPO Algorithm and Parameters 7
5.2 Dynamic Exploration Schedule 7
5.3 Training Loop and Data Collection 7
5.4 Returns and Advantage Estimation 8
5.5 Policy Update with PPO 8
5.6 Progress Monitoring and Early Stopping 9

6 Results **10**

7 Conclusion **13**

8 Sources **13**

1 Introduction

This project implements and evaluates a reinforcement learning (RL) agent acting in a continuous, two-dimensional environment. The agent's task is to learn to navigate efficiently towards a goal whose position changes between episodes.

The central goal of the work is to train a policy in a simulated environment whose physical parameters are randomized — mass, friction, sensor noise, and the goal's position — in order to study how well the learned policy generalizes to new, unseen parameter settings. Evaluation focuses specifically on *zero-shot transfer*: whether the trained policy can cope with conditions it never encountered during training, without any further fine-tuning.

The problem is addressed with Proximal Policy Optimization (PPO), one of the most widely used and reliable policy-gradient methods. The agent is implemented as an Actor–Critic neural network, which predicts both the policy (the distribution over actions) and an estimate of the state value.

The environment was built entirely from scratch, without relying on existing RL-environment libraries, and includes realistic physical characteristics such as mass, friction, and a simulated LiDAR-type sensor. Particular care was given to both functionality and visual rendering (2D and 3D), so that the agent's behaviour could be inspected and analyzed directly.

2 Domain Randomization

Domain randomization is a technique used primarily in reinforcement learning and robotics to improve an agent's generalization. The core idea is to inject deliberate randomness into the training environment: every episode is run with different physical parameters, such as mass, friction, the goal's position, sensor noise level, or even a different obstacle layout.

In this way, the agent is trained across a wide variety of scenarios instead of adapting to a single, fixed, constrained environment. This improves robustness and allows the policy to cope satisfactorily even with situations it has never seen before (zero-shot generalization).

The technique is especially valuable when there is a gap between simulation and the real world (the *sim-to-real gap*): by randomizing the simulated domain, the policy trained in simulation is more likely to transfer successfully to real-world conditions, where the true parameters are uncertain or variable.

In this project, domain randomization was applied to the agent's mass, its friction coefficient, the goal's layout, and the characteristics of the LiDAR-type sensor. This strengthened the policy's ability to operate reliably under different, unpredictable conditions.

3 Environment

3.1 Environment Initialization

At initialization, the environment defines the core state and action spaces, together with the physical characteristics governing the agent's dynamics. The environment is continuous and two-dimensional, with the agent moving on the xy-plane, within the range $[-10, 10] \times [-10, 10]$.

The observation space includes both physical and geometric parameters: specifically, the relative position of the goal (obtained via the simulated LiDAR), the agent's mass, and its friction coefficient.

The agent's action is defined as a two-dimensional force that can be applied in any direction. The magnitude of this force is bounded so that it stays within physically reasonable limits; this design gives the learned policy more flexibility compared to a discrete action space.

The goal's initial position is randomized, while the agent always starts from the origin. The choice of specific parameters — mass, friction, and the timestep Δt — uses realistic physical

magnitudes and forms the basis for the domain randomization applied later, at each call to `reset()`.

Finally, initialization also prepares the variables needed for 2D visualization of the environment, allowing the agent's progress to be monitored as training proceeds (Figure 1).

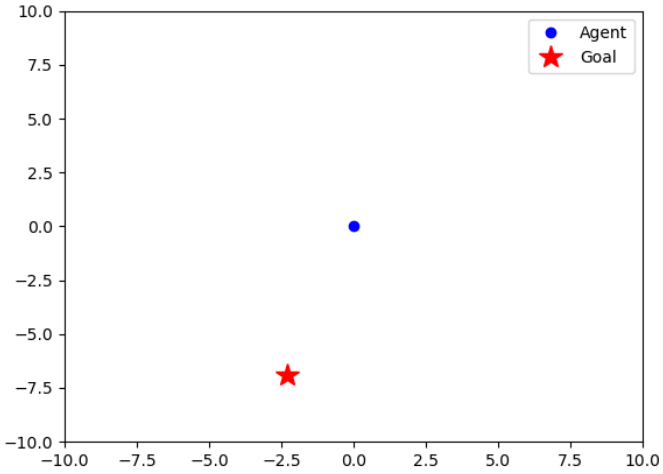


Figure 1: Initial state of the environment: the agent (blue) starts at the origin, the goal (red star) is placed at a randomized position within the $[-10, 10] \times [-10, 10]$ workspace.

3.2 Dynamics

The system takes as input the force applied to the agent. The next state of the system is computed in the `compute_dynamics` function, which implements the basic laws of dynamics to determine the agent's new position and velocity:

$$\text{Direction of the velocity vector:} \quad \hat{\mathbf{d}} = \frac{\mathbf{v}}{\|\mathbf{v}\| + 10^{-8}} \tag{1}$$

$$\text{Friction force:} \quad \mathbf{f}_{\text{friction}} = -\,\text{friction} \cdot \hat{\mathbf{d}} \tag{2}$$

$$\text{Acceleration:} \quad \mathbf{a} = \frac{\mathbf{F} + \mathbf{f}_{\text{friction}}}{m} \tag{3}$$

$$\text{Velocity update:} \quad \mathbf{v}_{\text{new}} = \mathbf{v} + \mathbf{a} \cdot \Delta t \tag{4}$$

$$\text{Position update:} \quad \mathbf{x}_{\text{new}} = \mathbf{x} + \mathbf{v} \cdot \Delta t + \tfrac{1}{2}\mathbf{a} \cdot \Delta t^2 \tag{5}$$

where `friction` denotes the magnitude of the friction coefficient and m the agent's mass. The small constant 10^{-8} in the direction term avoids division by zero when the agent is at rest. This is a standard semi-implicit Euler integration scheme: friction is modeled as a constant-magnitude force opposing the current direction of motion (Coulomb-type friction), rather than a velocity-proportional drag term.

3.3 Goal Detection

For added realism, the agent is assumed not to know the goal's position directly; instead, it is equipped with a LiDAR-type sensor that scans the surrounding space.

The `lidar` function simulates the operation of a 2D LiDAR. Its purpose is to detect and measure the distance and bearing angle to fixed reference points (in this setup, only the goal) relative to the agent's position. To better simulate a real measuring device, Gaussian noise is added to the measurements.

For each landmark, if the distance is smaller than a maximum range (`max_range`), the detection is recorded. The result is a list of detections, each containing the distance, the angle, and the ID of the corresponding landmark:

Implementation note. What the code actually does. `self.landmarks` is set to `[tuple(self.goal)]` — a single-element list — both in `reset()` and after. So, despite the general "LiDAR scanning the surrounding space" framing, this sensor never detects anything other than the goal itself; there are no other landmarks or obstacles to sense. Likewise, every call site (`reset()`, `step()`) calls `lidar(self.position, self.landmarks, noise_std=0.01)` without passing `max_range`, so it keeps the function's default, `max_range=np.inf` — the range check described above is present in the code but, as actually used here, never excludes a detection. The sensor is therefore, in this project, equivalent to a noisy relative bearing-and-range sensor to the goal, not a general-purpose LiDAR.

$$d_{\text{noisy}} = d + \mathcal{N}(0, \sigma^2) \tag{6}$$

$$\theta_{\text{noisy}} = \theta + \mathcal{N}(0, \sigma^2) \tag{7}$$

where $d = \|(m_x, m_y) - (p_x, p_y)\|$ and $\theta = \text{atan2}(m_y - p_y,\, m_x - p_x)$ are the noise-free distance and bearing to a landmark at (m_x, m_y) from the agent's position (p_x, p_y).

3.4 Reward Computation

The reward signal is a critical component for successfully training the agent, as it shapes the learned behaviour towards the desired direction. The implemented reward function combines three main terms:

Negative distance term. The main driving term is the negative distance to the goal ($-$ distance), which continuously encourages the agent to reduce its Euclidean distance to the destination. This term provides steady guidance towards the correct direction.

Directional alignment term. To promote efficient navigation, the reward is boosted by a positive term computed as the inner product between the normalized action and the normalized direction towards the goal. This term takes values in $[-1, 1]$ and rewards actions that are aligned with the desired direction of motion.

Success bonus. When the agent comes within the goal's critical radius (the `threshold`), it receives a substantial bonus of $+100$. This signals successful completion of the episode and reinforces the behaviour that led to it.

Written out explicitly, with $\hat{\mathbf{a}}$ the normalized action and $\hat{\mathbf{g}}$ the normalized direction towards the goal:

$$r = -\,\|\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{agent}}\| \;+\; 10\,(\hat{\mathbf{a}} \cdot \hat{\mathbf{g}}) \;+\; \mathbf{1}[\|\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{agent}}\| < \text{threshold}] \cdot 100. \tag{8}$$

Equation (8) matches exactly what `compute_reward` computes in `env.py`.

3.5 Agent–Environment Interaction

The `step` function implements the core logic of the agent's interaction with the environment at each timestep. Specifically, it takes an action (a force) as input and returns the observation, the reward, and a flag indicating episode termination (`done`).

First, the action is clipped to the allowed range. The resulting force is then passed to `compute_dynamics` to update the agent's position and velocity, accounting for the system's physics. The position is clipped so that it stays within the predefined workspace bounds.

Next, the LiDAR sensor is triggered to detect the goal. If a valid detection occurs, the distance and angle to the goal are extracted and converted to Cartesian components (dx, dy).

These, together with the agent's physical parameters (mass and friction), make up the observation vector returned to the agent.

Immediately afterward, the reward is computed via `compute_reward`, as described above. The episode terminates either when the agent reaches the goal or when the maximum number of steps is exhausted.

3.6 3D Visualization

To monitor the agent's training, a 3D visualization system was implemented using the `pygame` and `OpenGL` libraries.

The function `init_pygame_3d()` creates a 1000×800 window with OpenGL support. A perspective projection is applied (field of view 45°, depth range 0.1–100), and the camera is positioned so that the two-dimensional plane is viewed as a 3D "scene."

Lighting is enabled via `GL_LIGHT0`, with the light source placed at $(10, 10, 20)$ and configured ambient, diffuse, and specular parameters. Objects are given materials with appropriate reflectivity and shininess properties for a more realistic appearance.

A space-themed background is loaded with `pygame` and converted into an OpenGL texture. It is rendered by `draw_background()`, which uses an orthographic projection and temporarily disables lighting and depth testing (Figure 2 shows this background, together with the agent and goal rendered as cylinders on the ground plane).

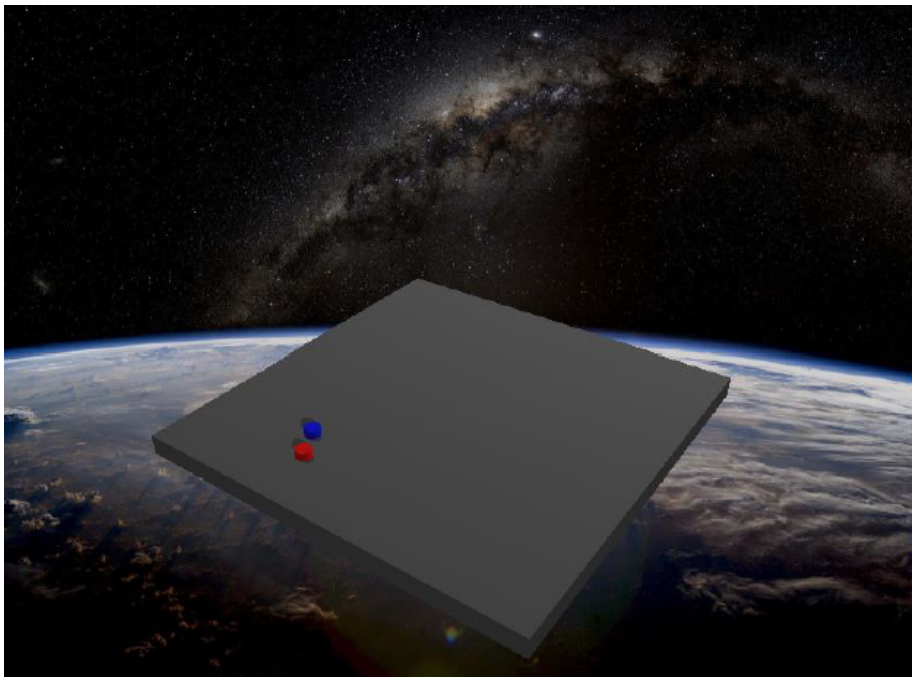


Figure 2: 3D visualization of the environment: the ground plane is rendered as a platform against a space-themed background, with the agent and goal shown as colored cylinders.

The environment includes:

- **Ground.** `draw_ground()` builds a ground plane with extent $[-10, 10]$ and a height of 1 unit, from quads with correctly oriented normal vectors.
- **Cylinders.** The agent and the goal are each rendered as a cylinder of radius and height 0.4, via `draw_cylinder()` and the GLU library.

`draw_shadow()` creates semi-transparent shadows, rendered as gray disks slightly above the ground plane to avoid z-fighting. Each object is given distinct material properties:

- **Agent:** blue $(0.1, 0.1, 1.0)$, shininess 50.0.
- **Goal:** red $(1.0, 0.1, 0.1)$, shininess 30.0.
- **Ground:** gray-blue $(0.3, 0.35, 0.45)$, shininess 10.0.

`render_3d()` brings all of this together: clearing the buffers, drawing the background, the ground, the shadows, and the objects, and finally updating the display buffer with a refresh period of 10 ms. In the training loop, this visualization is only triggered during the final 5% of training epochs (every 5 steps), so as not to slow down the bulk of training.

4 Neural Network Architecture

4.1 Actor–Critic Model

To solve the navigation problem, an Actor–Critic architecture was adopted, which combines the advantages of value-based methods and policy-based methods. The model consists of shared layers and two specialized heads: the *Actor*, which produces the policy's action, and the *Critic*, which estimates the state's value.

The shared architecture consists of a linear layer mapping the observation (`obs_dim` $= 4$) to a 64-dimensional representation, followed by a tanh non-linearity. This shared representation serves both policy learning and value estimation.

The Actor consists of two linear layers with an intermediate tanh activation, producing logits for the two action dimensions (`act_dim` $= 2$). The Actor's output is used as the mean of a Normal distribution that models the policy's action. The Critic follows a similar structure, producing a single value representing the long-term expected return (the estimated value of the current state).

$$h = \tanh(W_1 o + b_1), \tag{9}$$

$$\mu = W_{a2}\big(\tanh(W_{a1} h + b_{a1})\big) + b_{a2}, \tag{10}$$

$$V(s) = W_{c2}\big(\tanh(W_{c1} h + b_{c1})\big) + b_{c2}, \tag{11}$$

where $o \in \mathbb{R}^4$ is the observation, $h \in \mathbb{R}^{64}$ the shared representation, μ the mean of the action distribution and $V(s)$ the Critic's value estimate.

Implementation note. `env.py` declares `self.observation_space = spaces.Box(...)` with 6 bounds (it includes the agent's raw (x, y) position and velocity-related bounds), so `env.observation_space.shape[0]` is actually 6, not 4. `main.py` computes `obs_dim = env.observation_space.shape[0]` but then does not use that variable: it instantiates `ActorCritic(obs_dim=4, act_dim=2)` with a hardcoded 4, which happens to match what `reset()`/`step()` actually return (Sec. 4.2) but not the environment's own declared observation space. This is a real inconsistency between the Gymnasium `observation_space` contract and the actual observation shape; the network works because the hardcoded constant was set correctly by hand, not because the two are kept consistent in code.

4.2 Observation Space

The observation vector was designed to give the agent the information needed for effective navigation. Specifically, it consists of four dimensions:

- **Relative goal position** (dx, dy)**.** The Cartesian components of the relative goal position, obtained by converting the LiDAR's polar measurements: $dx = d\cos\theta$, $dy = d\sin\theta$.
- **Agent mass.** Affects the system's dynamic behaviour according to the laws of motion; including it lets the agent adapt its strategy to its own physical characteristics.

- **Friction coefficient.** Determines the resistance to motion, affecting the agent's dynamic response to applied forces.

The choice of this compact observation space follows the principle of minimizing complexity while retaining the necessary information. Rather than providing the absolute coordinates of the agent and the goal directly, the relative position offers translation invariance, while the physical parameterization allows the policy to adapt to different dynamic conditions.

5 Training with Proximal Policy Optimization (PPO)

5.1 PPO Algorithm and Parameters

To train the Actor–Critic model, the Proximal Policy Optimization (PPO) algorithm was chosen, which combines the advantages of trust-region methods with simplicity of implementation, using clipping on the policy ratio to avoid excessively large updates.

Two configurations were trained and compared in this project, differing only in the total number of epochs (see Section 6):

- **Learning rate:** 1×10^{-4}, with exponential decay (a `StepLR` scheduler, $\gamma = 0.8$ every 10 epochs)
- **Steps per epoch:** 1024 interaction steps per epoch
- **Mini-batch size:** 64 samples per mini-batch
- **Clip epsilon:** 0.1, for clipping the policy ratio
- **Discount factor (γ):** 0.99, used when computing returns
- **GAE λ:** 0.95, for Generalized Advantage Estimation
- **Total epochs:** 100 in the first configuration, 200 in the second.

5.2 Dynamic Exploration Schedule

An important feature of the implementation is the dynamic adjustment of exploration during training. The standard deviation of the action distribution decreases linearly from 0.5 at the start of training to 0.1 at the end, according to:

$$\sigma = \sigma_{\text{initial}} - \left(\sigma_{\text{initial}} - \sigma_{\text{final}}\right) \cdot \frac{\text{epoch}}{\text{total_epochs}}. \tag{12}$$

This schedule allows the agent to explore the action space extensively in the early stages, while gradually "trusting" the network's output more as the policy evolves and improves.

5.3 Training Loop and Data Collection

During the main training loop, the agent interacts with the environment for 1024 steps per epoch, collecting trajectories of experience. During this phase, the following are stored:

- **Observations** (`obs_buf`): the environment states.
- **Actions** (`act_buf`): the agent's selected actions.
- **Log-probabilities** (`logp_buf`): the actions' probabilities under the policy that collected them.

- **Value estimates** (`val_buf`): the Critic's predictions.
- **Rewards** (`rew_buf`): the rewards received.
- **Done flags** (`done_buf`): the episode-termination indicators.

Data collection is accompanied by a success counter (`success_count`), incremented whenever the agent reaches within the goal's critical radius.

5.4 Returns and Advantage Estimation

Once the data has been collected, returns are computed using the Generalized Advantage Estimation (GAE) algorithm. GAE is a technique that addresses the classic bias–variance trade-off in advantage estimation:

- **Monte Carlo returns:** high variance, low bias.
- **Temporal-Difference (TD) methods:** low variance, higher bias.
- **GAE:** combines both approaches through a parameter λ ($\lambda = 1$ corresponds to MC, $\lambda = 0$ to TD).

For each timestep, a TD error (δ) is computed, expressing the discrepancy between the actual reward received and the model's prediction:

$$\delta_t = r_t + \gamma\, V(s_{t+1})\,(1 - d_t) - V(s_t), \tag{13}$$

where $d_t = 1$ if the episode terminated at step t (`done`), and 0 otherwise (so that the bootstrap term vanishes at an episode boundary, correctly reducing to $\delta_t = r_t - V(s_t)$).

GAE then uses this error with exponential smoothing (via the hyperparameters γ and λ) to produce the final advantage estimate, computed recursively backward from the end of the trajectory to its start:

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\,\hat{A}_{t+1}. \tag{14}$$

Unrolling the recursion gives the equivalent, more commonly cited closed form as a weighted sum of future TD errors:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l\, \delta_{t+l}. \tag{15}$$

The returns used as the Critic's regression target are then:

$$\text{return}_t = \hat{A}_t + V(s_t). \tag{16}$$

Advantages are normalized by subtracting their mean and dividing by their standard deviation, a technique that improves training stability and accelerates convergence:

$$\hat{A}_t \leftarrow \frac{\hat{A}_t - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + \varepsilon}. \tag{17}$$

5.5 Policy Update with PPO

Training is driven by the advantage, which expresses the discrepancy between the return actually achieved and the value predicted by the Critic. A positive advantage means the outcome was better than expected, so the agent should reinforce that action's probability; a negative advantage implies underperformance, so the action's probability should be reduced.

PPO implements these updates while avoiding large changes that could destabilize training. To this end, it first computes the probability ratio between the new and the old policy:

$$\text{ratio} = \frac{\pi_{\text{new}}(a \mid s)}{\pi_{\text{old}}(a \mid s)} = \exp\big(\log \pi_{\text{new}} - \log \pi_{\text{old}}\big). \tag{18}$$

This ratio shows how much the action's probability has changed: a ratio > 1 means the new policy considers the action more likely, while a ratio < 1 means less likely.

The **PPO clipped objective** uses two versions of the surrogate loss:

$$\text{surrogate}_1 = \text{ratio} \cdot \hat{A}, \tag{19}$$

$$\text{surrogate}_2 = \text{clip}(\text{ratio}, 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}. \tag{20}$$

Implementation note. Actions during rollout collection are sampled from $\mathcal{N}(\mu, \sigma^2_{\text{epoch}})$ with the annealed σ_{epoch} of Sec. 5.2 (Eq. 8). When recomputing $\log \pi_{\text{new}}$ for the ratio above, however, `main.py` builds the "new" distribution as `Normal(logits, torch.tensor(0.2))` — a fixed standard deviation of 0.2, independent of the epoch. Except at the one epoch where $\sigma_{\text{epoch}} = 0.2$, the distribution used to *act* and the distribution used to *evaluate the action's new log-probability* are therefore not the same distribution, which is a slight inconsistency in how the importance ratio is defined (early on, $\sigma_{\text{epoch}} > 0.2$ so π_{new} is evaluated as narrower than the policy that actually acted; later, $\sigma_{\text{epoch}} < 0.2$ and it is evaluated as wider). In practice this mainly affects the scale of the ratio rather than its sign, so training still worked, but using the same σ_{epoch} in both places would be the more principled choice.

During training, the experience gathered from the rollout is shuffled randomly and split into mini-batches of size 64. The update is applied on each mini-batch and repeated 10 times per epoch to achieve stable convergence. In each such pass, three loss terms are computed:

Policy loss:

$$L_{\text{policy}} = -\min(\text{surrogate}_1, \text{surrogate}_2). \tag{21}$$

The negative sign is used because the network minimizes its loss, i.e. it maximizes the surrogate objective (reinforcing good actions). Hence, the more negative the policy loss, the better the policy update is considered to be.

Value loss: the mean squared error between the value estimates and the computed returns,

$$L_{\text{value}} = \frac{1}{N} \sum_{i=1}^{N} \big(\text{return}_i - V(s_i)\big)^2. \tag{22}$$

Entropy: measures the policy's uncertainty. High entropy means the policy spreads probability roughly uniformly across the available actions, while low entropy indicates decisiveness and reduced exploration. Encouraging entropy prevents premature convergence to local optima, promoting more resilient behaviour, especially in the early stages of training.

The total loss combines the three terms:

$$L = L_{\text{policy}} + 0.5\, L_{\text{value}} - 0.01\, \mathcal{H}, \tag{23}$$

where $\mathcal{H}$ denotes the policy's entropy. This procedure embodies the core principles of policy-gradient methods, offering a way to update the policy with a view toward both stability and training effectiveness.

5.6 Progress Monitoring and Early Stopping

Training is carried out over multiple epochs and monitored via the mean reward per epoch. An early stopping mechanism is implemented, with patience equal to 25% of the total number of epochs (50 epochs in the 200-epoch configuration). Whenever a new best model is detected, its state dictionary is saved for later use.

During the final 5% of epochs, the 3D visualization is enabled every 5 steps, allowing visual inspection of the trained agent's behaviour. Training ends either when all epochs are completed or when early stopping is triggered, ensuring efficient use of computational resources and helping to avoid overfitting.

Figure 3 shows an excerpt of the console output during one of the training runs, illustrating how the average reward per epoch evolves and when new best models are found.

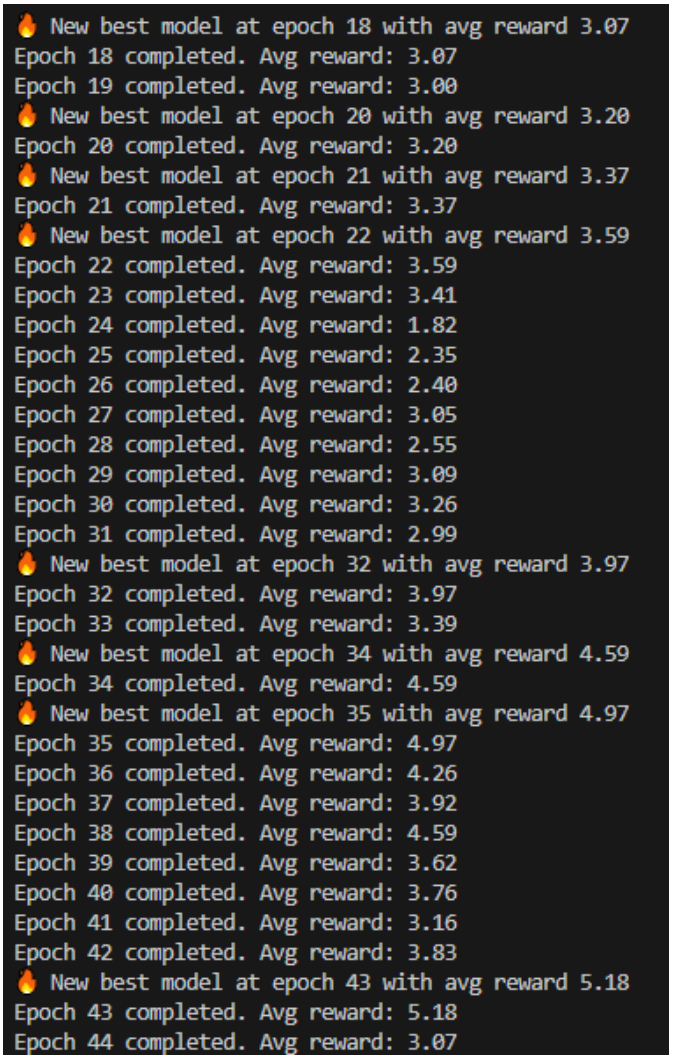


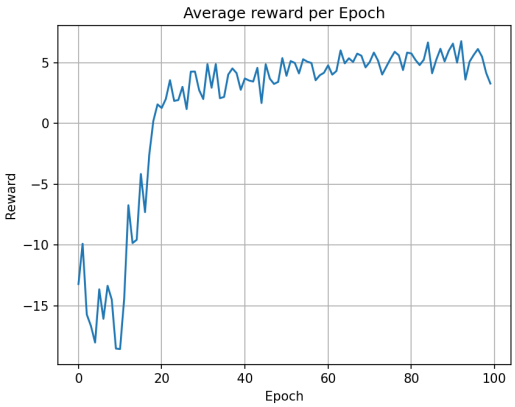
Figure 3: Console output during training: the average reward per epoch, with new best models flagged as they are found.

6 Results

As shown in the corresponding plots, the agent's policy improves progressively over the course of training. Figure 4 shows the evolution of the mean reward per epoch for the two trained configurations. A clear improvement trend is visible during training, with rewards stabilizing at positive values after the first 20–40 epochs, indicating that the agent has learned to approach its goal effectively.

```
🔥 New best model at epoch 18 with avg reward 3.07
Epoch 18 completed. Avg reward: 3.07
Epoch 19 completed. Avg reward: 3.00
🔥 New best model at epoch 20 with avg reward 3.20
Epoch 20 completed. Avg reward: 3.20
🔥 New best model at epoch 21 with avg reward 3.37
Epoch 21 completed. Avg reward: 3.37
🔥 New best model at epoch 22 with avg reward 3.59
Epoch 22 completed. Avg reward: 3.59
Epoch 23 completed. Avg reward: 3.41
Epoch 24 completed. Avg reward: 1.82
Epoch 25 completed. Avg reward: 2.35
Epoch 26 completed. Avg reward: 2.40
Epoch 27 completed. Avg reward: 3.05
Epoch 28 completed. Avg reward: 2.55
Epoch 29 completed. Avg reward: 3.09
Epoch 30 completed. Avg reward: 3.26
Epoch 31 completed. Avg reward: 2.99
🔥 New best model at epoch 32 with avg reward 3.97
Epoch 32 completed. Avg reward: 3.97
Epoch 33 completed. Avg reward: 3.39
🔥 New best model at epoch 34 with avg reward 4.59
Epoch 34 completed. Avg reward: 4.59
🔥 New best model at epoch 35 with avg reward 4.97
Epoch 35 completed. Avg reward: 4.97
Epoch 36 completed. Avg reward: 4.26
Epoch 37 completed. Avg reward: 3.92
Epoch 38 completed. Avg reward: 4.59
Epoch 39 completed. Avg reward: 3.62
Epoch 40 completed. Avg reward: 3.76
Epoch 41 completed. Avg reward: 3.16
Epoch 42 completed. Avg reward: 3.83
🔥 New best model at epoch 43 with avg reward 5.18
Epoch 43 completed. Avg reward: 5.18
Epoch 44 completed. Avg reward: 3.07
```

100 epochs

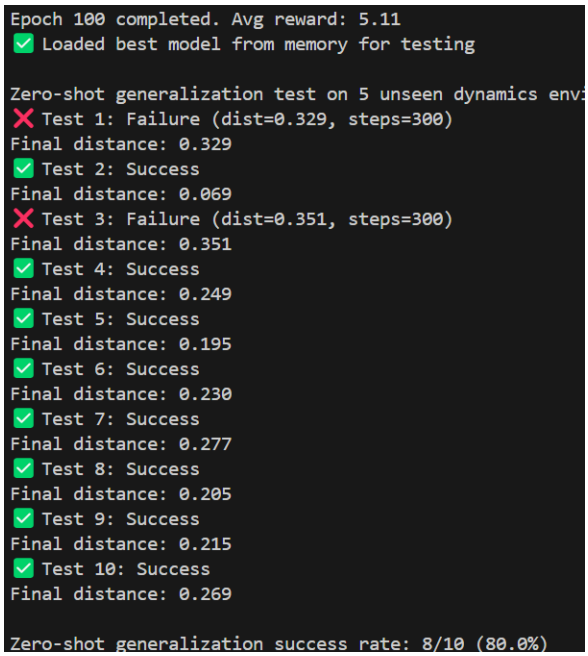


200 epochs

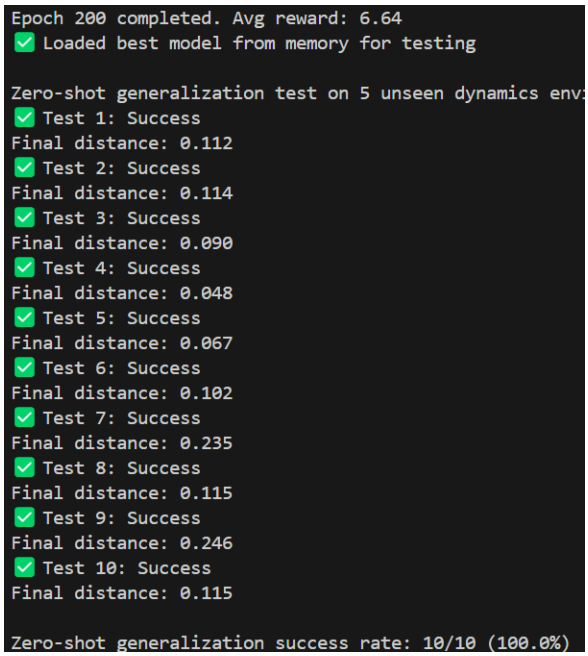
Figure 4: Average reward per epoch for the 100-epoch and 200-epoch training configurations.

After training, the best policy model is evaluated on 10 environments with randomized parameters, to assess zero-shot transfer. As shown in Figure 5, both policies perform quite well (a high success rate), with the second model, trained for 200 epochs, reaching a 100% success rate.

Implementation note. What "unseen" actually means here. Each test environment is a fresh `Point2DEnv()`, and `reset()` already randomizes mass ($\mathcal{U}(0.5, 1.0)$), friction coefficient ($\mathcal{U}(0, 0.1)$) and the goal position on every call — this is the real source of the domain-randomization tested here. `main.py`'s test loop also contains commented-out lines meant to push these test environments to a wider, explicitly "harder" range (`test_env.mass = np.random.uniform(1.8, 2.5)`, etc.), which would have made the test distribution genuinely different from, and wider than, the training distribution; as the code currently stands, those lines are disabled, so the 10 test episodes are drawn from *the same* parameter distribution as training, not a distinct held-out one. "Zero-shot" here is accurate in the sense that the specific mass/friction/goal values of each test episode were not seen during training (they are freshly sampled), but the claim is weaker than "generalizes beyond the training distribution" would suggest. Separately, the success criterion also differs between training and testing: `env.py`'s `reset()` sets `self.threshold = 0.1` (used to flag an episode as a success *during training*, Sec. 3.4), while the test loop in `main.py` uses `test_threshold = 0.3` (three times as lenient) to decide the success/failure shown in Figure 6. Re-enabling the wider test-time randomization, and/or re-testing with the stricter 0.1 threshold, would be a natural next step to get a more demanding measure of generalization.



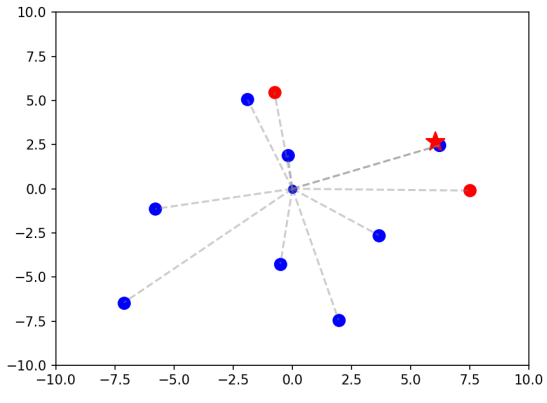
100-epoch model: 8/10 (80%)



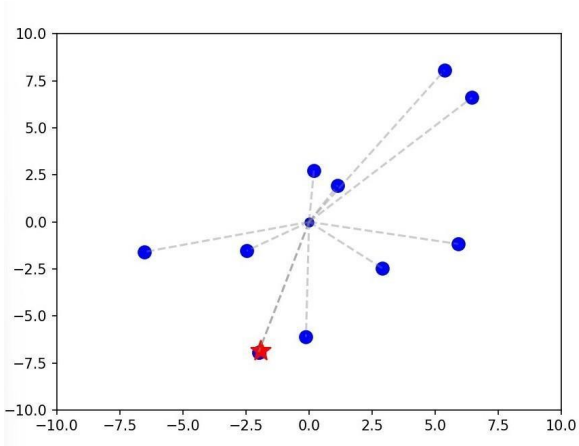
200-epoch model: 10/10 (100%)

Figure 5: Zero-shot generalization test on 10 unseen, randomized environments.

Figure 6 shows the results of the test runs in the two-dimensional environment. Agents that reached the goal within the threshold distance ($= 0.3$) are shown in blue, while those that did not are shown in red; the final goal position is also shown for each test.



100-epoch model (2 failures, shown in red)



200-epoch model (all successes)

Figure 6: Final agent positions across the 10 zero-shot test environments, relative to each episode's goal.

These results show that the policy managed to learn a navigation strategy that is stable and effective, even in randomized environments it had never encountered. The success on the zero-shot tests demonstrates that domain randomization contributed substantially to the agent's generalization, and that the additional 100 epochs of training (200 vs. 100) translated into a tangible, measurable improvement in zero-shot robustness (80% $\rightarrow$ 100% success, and a reduction in the final distance-to-goal spread across the successful runs).

Limitations and Possible Improvements

- **Single landmark, infinite range.** As noted in Sec. 3.3, the "LiDAR" only ever senses the goal itself and the range check is never active in practice; a version with real obstacles and a finite sensing range would be a meaningfully different (and harder) problem.
- **Test-time randomization range.** As noted in Sec. 6, the code to widen the test distribution beyond the training distribution exists but is disabled, so the reported zero-shot numbers test resampling, not out-of-distribution generalization in the stricter sense.
- `obs_dim`/`observation_space` **mismatch.** The environment's declared Gymnasium observation space (Sec. 4.1) does not match the observation actually returned; this works only because the network's input size was hardcoded correctly by hand.
- **Exploration/update std mismatch.** As noted in Sec. 5.5, the PPO update evaluates log-probabilities under a fixed-std Normal rather than the same annealed std used to act, which is a minor theoretical inconsistency in the importance-ratio computation.
- **No obstacle avoidance or multi-goal setting.** The task is reduced to point-to-point navigation with no obstacles between agent and goal; extending domain randomization to obstacle layout (mentioned as a general possibility in Sec. 2) was not actually implemented.

7 Conclusion

This project implemented a complete sim-to-sim domain randomization pipeline for a 2D navigation task, from the physics and sensor simulation up through a PPO-trained Actor–Critic policy and a 3D visualization layer, all built without relying on an existing RL-environment framework. Randomizing mass, friction, and goal placement at each episode reset, combined with a compact, translation-invariant observation space, produced a policy that transfers in a zero-shot setting to previously unseen dynamics with a high success rate, and that success rate improves further with additional training. The main practical takeaway is that even a fairly simple domain-randomization scheme, applied to just a handful of physical parameters, is enough to produce meaningfully robust zero-shot generalization in this setting.

8 Sources

1. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. (2017, September). Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 23–30).
2. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms.* arXiv preprint arXiv:1707.06347. arxiv.org/abs/1707.06347
3. Weng, L. (2018). Policy Gradient Algorithms. lilianweng.github.io/posts/2018-04-08-policy-gradient
4. Course lecture slides (Intelligent Control).
5. ChatGPT was used for supporting research while documenting this project.